

Build a Production-Ready Streamlit Data Cleaning & Analysis Application

Develop a complete, functional **Streamlit web application for CSV and Excel data analysis, cleaning, visualization, and basic natural-language querying.**

The application must be fully functional end-to-end. Do **not** create placeholder buttons, dummy functionality, mock results, or UI elements that do not perform the stated operation.

1. Core Technology Requirements

Use Python and Streamlit.

Recommended libraries:

- `streamlit`
- `pandas`
- `numpy`
- `openpyxl` for Excel `.xlsx` files
- `matplotlib`
- `seaborn`
- `plotly`
- A local deterministic query/parsing library such as `duckdb` where useful
- Standard Python libraries where appropriate

Important constraints

- **No OpenAI API**
- **No external AI/LLM API**
- **No cloud AI service**
- The application must work locally.
- Uploaded data must be processed locally.
- Do not send uploaded datasets to external services.
- Natural-language querying must use a deterministic/local approach rather than an AI model.

2. File Upload

Create a clear file-upload section.

The user must be able to upload:

- `.CSV`

- `.xlsx`
- `.xls` if supported by the chosen dependency

Use Streamlit's file uploader.

After upload:

1. Detect the file type.
2. Load it into a pandas DataFrame.
3. Store the DataFrame in `st.session_state`.
4. Display the loaded filename.
5. Display a preview of the dataset.
6. Handle invalid or corrupted files gracefully with a useful error message.

For Excel files:

- If multiple sheets exist, allow the user to select the sheet to analyze.
- Show the selected sheet name.

3. Dataset Summary

After successfully loading a dataset, display a professional dataset summary.

At minimum show:

- Number of rows
- Number of columns
- Number of missing values
- Number of duplicate records
- Memory usage if available
- Number of numeric columns
- Number of object/string columns
- Number of datetime columns

Use Streamlit metrics/cards where appropriate.

Also provide a column-level summary containing:

Column	Data Type	Non-Null Count	Missing Count	Missing %	Unique Values
--------	-----------	----------------	---------------	-----------	---------------

The summary must update whenever the dataset changes.

4. Data Preview

Provide a dedicated data preview section.

Requirements:

- Display the complete dataset in an interactive Streamlit dataframe.
- Allow the user to inspect the first/last records.
- Show the current number of rows and columns.
- After every cleaning operation, refresh the displayed dataset and statistics.

Do not create a second independent copy of the dataset that can become inconsistent with the cleaned dataset.

Use one authoritative DataFrame in `st.session_state`.

5. Data Cleaning Module

Create a dedicated **Data Cleaning** section.

Every cleaning operation must modify the working DataFrame only after the user explicitly chooses the operation.

Provide an undo/reset mechanism where practical.

5.1 Remove Rows Containing Missing Values

Provide an option:

Remove rows containing missing values

Allow the user to choose whether to:

- Remove rows containing any missing value
- Remove rows only when all values are missing
- Optionally remove rows based on selected columns

Before applying the operation, display:

- Rows before
- Rows that will be removed
- Rows remaining

Require an explicit confirmation/action button.

6. Duplicate Record Cleaning

Provide a **Duplicate Records** section.

Show:

- Number of duplicate rows
- Number of unique rows

Allow the user to:

- Remove duplicate records
- Choose whether duplicate detection uses all columns or selected columns
- Choose which duplicate occurrence to keep:
 - First
 - Last

Display the expected number of removed rows before applying the operation.

After applying the operation, update all dataset statistics.

7. Missing Value Handling by Data Type

Create a dedicated **Missing Value Treatment** interface.

The available methods must depend on the selected column's data type.

7.1 Object/String Columns

For object/string/category-like columns, allow the user to fill missing values using:

- A custom value
 - "Unknown"
 - "Not Available"
 - "Missing"
- Mode

The implementation should use pandas-compatible operations such as `fillna()`.

Allow the user to select:

- One column
- Multiple columns

- All eligible object/string columns

Do not apply numeric interpolation methods to object columns.

8. Numeric Missing Values

For integer and floating-point columns, provide the following missing-value strategies:

Forward Fill

Use:

```
df[column].ffill()
```

Backward Fill

Use:

```
df[column].bfill()
```

Interpolation

Use pandas interpolation:

```
df[column].interpolate()
```

Provide a UI allowing the user to:

- Select one numeric column
- Select multiple numeric columns
- Apply the selected strategy

Show:

- Missing values before
- Missing values after
- Number of values filled

Make sure interpolation is applied only to appropriate numeric columns.

9. Additional Numeric Cleaning

Provide optional basic numeric cleaning functionality where practical:

- Convert compatible numeric-looking columns to numeric
- Handle invalid numeric values safely
- Allow the user to inspect numeric columns before conversion

Do not silently change column data types without telling the user.

10. Date/Time Handling

Detect columns that can reasonably be interpreted as dates.

Provide an optional conversion feature:

Convert selected column to datetime

Use safe pandas conversion with error handling.

Do not automatically convert every string column to datetime because this can corrupt legitimate string data.

11. Download Cleaned Dataset

Provide a prominent **Download Cleaned Dataset** section.

The user must be able to download the currently active cleaned DataFrame.

Support:

- CSV download
- Excel download

The downloaded file must contain the actual current DataFrame after all transformations performed by the user.

The filename should be something like:

```
cleaned_dataset.csv
```

or

cleaned_dataset.xlsx

Do not generate an empty or stale download.

12. Natural-Language Data Questions

Create a dedicated section:

Ask Questions About Your Data

The user should be able to type basic questions in natural language.

Examples:

- sales by region
- region per sales
- best selling products
- top 10 products by sales
- average sales by region
- total sales by category
- count of customers by region
- maximum sales by product
- minimum sales by product
- show sales by month
- which region has the highest sales?

Critical Requirement

There must be:

NO API CALLS and NO AI/LLM dependency.

The implementation must work entirely locally.

Use a deterministic query engine/parser.

A suitable architecture is:

1. Detect the user's intent.
2. Identify relevant columns from the DataFrame.
3. Map recognized phrases to deterministic pandas/DuckDB operations.
4. Execute the generated operation locally.
5. Display the result as a table.
6. Where appropriate, provide a visualization of the result.

13. Natural-Language Query Engine Design

Do not attempt to support unlimited natural language.

Instead, implement a clearly defined set of supported analytical intents.

The parser should support concepts such as:

Aggregations

- sum
- total
- average/mean
- minimum
- maximum
- count
- number of records

Grouping

Examples:

- sales by region
- revenue by category
- quantity by product
- customers by country

Ranking

Examples:

- best selling products
- top 10 products by sales
- bottom 5 products by revenue

Filtering

Examples:

- sales in India
- products where sales > 1000

Time analysis

Where a valid datetime column exists:

- sales by month
- sales by year
- sales by day

The parser should normalize common language variations.

For example:

- region per sales
- sales per region
- sales by region

should be interpreted consistently.

14. Column Detection

The natural-language engine should inspect the actual DataFrame columns.

For example, if the dataset contains:

```
Region
Product
Sales
Quantity
Customer
```

then the query:

```
best selling products
```

should identify:

- Product → grouping column
- Sales → metric column

The engine must NOT assume that columns are always named exactly `Sales`, `Region`, or `Product`.

Implement reasonable column-name matching using:

- lowercase normalization

- whitespace normalization
- common synonyms
- fuzzy matching where appropriate

Examples of synonyms:

```
sales:
sales, sale, revenue, amount, turnover

quantity:
quantity, qty, units, volume

region:
region, area, territory, zone

product:
product, item, product_name, item_name
```

If the engine cannot confidently identify the required columns, it should ask the user to select the relevant columns rather than guessing incorrectly.

15. Query Result Presentation

For every successfully executed query:

1. Display the interpreted query/action.
2. Display the resulting DataFrame/table.
3. Display useful metrics where appropriate.
4. Provide an option to visualize the result.

Example:

User:

```
best selling products
```

Application:

```
Interpretation:
Group by: Product
Metric: Sales
Operation: Sum
```

Sort: Descending
Limit: 10

Then show:

Product	Total Sales
Product A	125000
Product B	110000
...	...

Do not claim that a query was understood if the parser failed.

16. Visualization Module

Create a dedicated **Visualization** section.

The user must be able to generate charts from the currently active cleaned dataset.

Support at minimum:

Matplotlib

- Bar chart
- Line chart
- Histogram
- Scatter plot
- Pie chart where appropriate

Seaborn

- Bar plot
- Count plot
- Histogram/distribution plot
- Box plot
- Scatter plot
- Heatmap

Plotly

- Interactive bar chart
- Line chart
- Scatter plot
- Pie/donut chart

- Histogram
 - Box plot
-

17. Visualization Controls

Create a dynamic visualization interface.

Allow the user to select:

Chart library

- Matplotlib
- Seaborn
- Plotly

Chart type

The available chart types should depend on the selected library.

X-axis

Select a DataFrame column.

Y-axis

Select a DataFrame column where applicable.

Aggregation

Where applicable:

- Sum
- Mean
- Count
- Min
- Max

Optional controls

- Chart title
- Sort ascending/descending
- Top N
- Figure height
- Figure width where applicable

The application should validate incompatible combinations.

For example:

- A categorical column can be used as a grouping axis.
- A numeric column can be used as a quantitative axis.
- Histogram should generally use a numeric column.
- Scatter plot should use numeric X/Y columns.

Display clear validation messages instead of throwing Python exceptions.

18. Visualization From Natural-Language Queries

Where possible, after executing a natural-language query such as:

```
total sales by region
```

provide:

Visualize Result

The application should automatically suggest an appropriate chart such as a bar chart.

The user should still have control over the chart configuration.

19. Session State

Use `st.session_state` correctly.

At minimum maintain:

- Original uploaded DataFrame
- Current working DataFrame
- Uploaded filename
- Selected Excel sheet
- Cleaning history if undo functionality is implemented
- Query results where useful

The user must be able to perform multiple cleaning operations sequentially without losing previous changes.

Example:

1. Upload dataset
2. Remove duplicates

3. Fill object missing values
4. Interpolate numeric missing values
5. Remove remaining missing rows
6. Analyze cleaned data
7. Generate charts
8. Download cleaned data

Every step must operate on the latest working DataFrame.

20. Reset Functionality

Provide a clear:

Reset Dataset

button.

When selected, restore the working DataFrame to the original uploaded dataset.

Require confirmation where appropriate.

After resetting:

- Recalculate statistics
 - Clear stale query results
 - Refresh the preview
 - Refresh missing-value counts
 - Refresh duplicate counts
-

21. Error Handling

The application must not crash because of normal user mistakes.

Handle:

- Invalid CSV files
- Invalid Excel files
- Empty files
- Empty DataFrames
- Missing required columns
- Unsupported chart combinations
- Invalid numeric conversion
- Invalid date conversion

- Query parser failures
- Queries referring to columns that do not exist
- Interpolation on unsupported data
- Download errors

Use appropriate Streamlit messages:

- `st.error()`
- `st.warning()`
- `st.info()`
- `st.success()`

Do not expose raw stack traces to normal users.

22. User Interface

Create a clean professional interface.

Suggested navigation:

Sidebar

- Upload Dataset
- Dataset Overview
- Data Cleaning
- Ask Questions
- Visualization
- Download

Use Streamlit tabs or navigation sections where appropriate.

The UI should clearly communicate:

- Current dataset
- Current row count
- Current column count
- Current missing values
- Current duplicate count

Use a professional dashboard-like layout.

23. Data Integrity

The application must never silently modify the dataset.

Every transformation should be triggered by an explicit user action.

After each transformation, display a small change summary such as:

```
Operation completed

Rows before: 10,000
Rows after: 9,850
Rows removed: 150
Missing values before: 420
Missing values after: 0
```

Where applicable.

24. Code Architecture

Do not put the entire application into one giant function.

Use a maintainable structure.

A suggested project structure:

```
project/
|
|— app.py
|— requirements.txt
|
|— modules/
|   |— __init__.py
|   |— data_loader.py
|   |— data_summary.py
|   |— data_cleaner.py
|   |— query_engine.py
|   |— visualizations.py
|   |— utils.py
```

If a simpler architecture is more appropriate, it is acceptable to use fewer files, but functionality should remain modular and maintainable.

25. Requirements.txt

Generate a complete `requirements.txt`.

Include all packages actually required by the implementation.

At minimum consider:

```
streamlit
pandas
numpy
openpyxl
matplotlib
seaborn
plotly
duckdb
```

Only include packages that are actually used by the final implementation.

Pin compatible versions where appropriate.

26. Testing and Validation — Mandatory

Before presenting the final code, thoroughly validate the implementation logically and, where possible, by executing tests.

Do NOT simply generate code and assume that it works.

Test the following scenarios:

Upload Tests

- CSV upload
- XLSX upload
- Empty dataset
- Invalid file

Summary Tests

- Correct row count
- Correct column count
- Correct missing-value count
- Correct duplicate count

Cleaning Tests

- Remove missing rows
- Remove duplicate rows
- Object `fillna`
- Numeric forward fill
- Numeric backward fill
- Numeric interpolation
- Multiple sequential cleaning operations
- Reset to original dataset

Query Tests

Test at least:

```
sales by region
region per sales
best selling products
top 10 products by sales
average sales by region
total sales by category
```

Test with different column naming conventions.

Visualization Tests

Test:

- Matplotlib
- Seaborn
- Plotly

Test multiple chart types and invalid combinations.

Download Tests

Verify that:

- CSV download contains the current cleaned DataFrame.
- Excel download contains the current cleaned DataFrame.

27. Final Deliverables

The final response must provide:

1. Complete working application source code.
2. `requirements.txt`.
3. Any additional Python modules required by the application.
4. Project directory structure.
5. Commands to install dependencies.
6. Command to run the application.

Example:

```
pip install -r requirements.txt
streamlit run app.py
```

Do **not** create a README file.

28. Final Quality Requirements

Before providing the final code, verify that:

- There are no undefined functions.
- There are no missing imports.
- There are no placeholder implementations.
- Every button performs its intended operation.
- Session state is handled correctly.
- Cleaning operations modify the correct DataFrame.
- Dataset statistics update after cleaning.
- Query results use the current cleaned dataset.
- Visualizations use the current cleaned dataset.
- Downloads use the current cleaned dataset.
- CSV and Excel uploads work.
- Excel downloads work.
- Natural-language querying does not require an API or AI model.
- The application does not send uploaded data to an external service.
- Normal user errors are handled gracefully.
- The application can be started using `streamlit run app.py`.

Most importantly, **prioritize correctness and working functionality over visual complexity**. If a feature cannot be implemented reliably, implement a clearly defined supported version rather than creating a misleading or non-functional interface.